Transformation

→ Resize = 128 x 128
→ ToTensor()
→ Normalization
mean = 0.5, 0.5, 0.5
Std = 0.5, 0.5, 0.5

RGB

Apple___healthy : 0

This class turns a **folder of many classes → numbered labels**
`folders → (image, class_id)`

Dataset Structure

```
root/
   Apple___healthy/
   Apple___scab/
   Tomato___late_blight/
   ...
```

Each folder = **one class**

🔥 What this code is doing

Step 1️⃣ : "List all classes"

```
self.classes = sorted([...])
```

👉 It creates:

```
[
'Apple___healthy',
'Apple___scab',
'Tomato___late_blight',
...
]
```

✓ Sorted = consistent order

Step 2️⃣ : "Give each class a number"

```
self.class_to_idx = {cls_name: idx ...}
```

👉 Creates a mapping:

```
Apple___healthy → 0
Apple___scab → 1
Tomato___late_blight → 2
```

```
...
```

Step 2️⃣ : "Go inside every folder"

```
for class_name in self.classes:
```

👉 For each class:

- open folder
- read all images

Step 4️⃣ : "Attach correct label to each image"

```
label = self.class_to_idx[class_name]
```

👉 So:

```
image in Apple___scab → label = 1
image in Tomato___late_blight → label = 2
```

Step 5️⃣ : "Store everything"

```
self.samples.append((img_path, label))
```

👉 Final structure:

```
[
("img1.jpg", 0),
("img2.jpg", 0),
("img3.jpg", 1),
("img4.jpg", 2),
...
]
```

What happens during training

When model asks:

```
image, label = dataset[i]
```

👉 `__getitem__` does:

1. Load image from disk
2. Convert to RGB
3. Apply transform
4. Return:
   `(image_tensor, class_id)`

Difference from your Binary version

Binary:

```
healthy → 0
everything else → 1
```

Multiclass:

```
every folder → its own unique number
```

Intuition

👉 This dataset **assigns a unique ID to each folder and tags every image with that ID**.

COMPLETE FLOW

1️⃣ Download Dataset

```
kagglehub → download PlantVillage dataset
```

👉 Output:

```
path → where dataset is stored
```

2️⃣ Setup Environment

```
- Fix randomness (seed)
- Select device (GPU / CPU)
```

👉 Model will run on:

```
cuda (if available) else cpu
```

3️⃣ Define Paths

```
TRAIN_PATH → train images folder
VAL_PATH   → validation images folder
```

👉 Structure:

```
train/
  Apple___healthy/
  Tomato___late_blight/
  ...
```

4️⃣ Define Transforms

```
Image → Resize → Tensor → Normalize
```

Flow:

```
PIL Image → (128×128) → Tensor → Normalized Tensor
```

5️⃣ Custom Dataset Class

What happens inside:

```
Step 1: Read all class folders
Step 2: Assign index to each class
Step 3: Loop through all images
Step 4: Store (image_path, label)
```

👉 Final:

```
self.samples = [
  (img_path1, label1),
  (img_path2, label2),
  ...
]
```

6️⃣ Load Dataset

```
train_dataset_full
test_dataset_full
```

👉 Also:

```
num_classes = total number of folders
```

7️⃣ Subset (Optional)

```
Take only first 100 samples
```

👉 Purpose:

```
Faster debugging / testing
```

8️⃣ DataLoader

```
Dataset → batches
```

Flow:

```
Dataset → DataLoader → batches of size 32
```

👉 Output per batch:

```
(batch_size, 3, 128, 128), (batch_size)
```

9️⃣ Model (CNN)

Feature extractor:

```
Conv → ReLU → BN → Pool (×3)
```

Shape flow:

```
(3,128,128)
→ (32,128,128)
→ (32,64,64)
→ (64,64,64)
→ (64,32,32)
→ (128,32,32)
→ (128,16,16)
```

Classifier:

```
Flatten → Linear → ReLU → Dropout → Linear → Output
```

Flow:

```
(128,16,16)
→ Flatten → 32768
→ 128 → 64 → num_classes
```

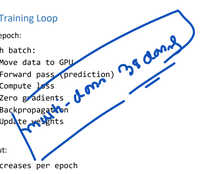
🔟 Training Setup

```
Loss → CrossEntropy
Optimizer → Adam
Epochs → 10
```

1️⃣1️⃣ Training Loop

For each epoch:

```
FOR each batch:
    1. Move data to GPU
    2. Forward pass (prediction)
    3. Compute loss
    4. Zero gradients
    5. Backpropagation
    6. Update weights
```



👉 Output:

```
Loss decreases per epoch
```

1️⃣2️⃣ Evaluation Function

```
model.eval()
no gradient calculation
```

Flow:

```
Input → Model → Predictions → Compare with labels → Accuracy
```

1️⃣3️⃣ Final Evaluation

```
Test Accuracy
```